

Finding the Limit: A Modular SLAM-to-Control Pipeline for F1TENTH Racing

Milan Manoj
University of Pennsylvania
milanmnj@seas.upenn.edu

Manasi Shrekhar
University of Pennsylvania
mshrek@seas.upenn.edu

Eric Ouyang
University of Pennsylvania
eouyang@seas.upenn.edu

Jackson Wang
University of Pennsylvania
yxjacksn@seas.upenn.edu

Abstract—Full autonomy in racing remains challenging due to real-time requirements in perception, localization, mapping, and control. This work presents a complete end-to-end pipeline for F1TENTH autonomous racing that requires no prior map or human-assisted initialization. The system performs a single exploratory lap using wall-following with simultaneous SLAM-based mapping via RTAB-Map, followed by automatic centerline extraction, raceline optimization, waypoint generation, and Pure Pursuit control for subsequent high-speed laps. An adaptive speed controller incrementally increases the target speed each lap and locks the maximum sustainable speed based on cross-track error feedback, eliminating manual speed tuning. The pipeline was validated across three distinct tracks—Levine, Towne, and Skirkanich—demonstrating robust generalization to unknown environments without any track-specific parameter changes.

I. INTRODUCTION

Fully autonomous vehicles are not yet capable of reliably operating in high-speed, unstructured environments such as racing. While substantial progress has been made in structured urban driving, racing introduces stricter constraints on latency, accuracy, and robustness. The F1TENTH platform [1] provides a controlled yet challenging setting in which to study these problems; however, most existing implementations rely on pre-built maps or human-assisted initialization, limiting their applicability to truly unknown environments.

This project targets full autonomy by removing these dependencies entirely. The core problem is to enable a vehicle to autonomously map, localize, plan, and control itself on an unknown track using only onboard sensing. This capability is important not only for competitive autonomous racing but also for broader real-world scenarios where prior maps are unavailable—such as exploration robotics, search-and-rescue operations, and emergency response vehicles.

The remainder of this paper is organized as follows. Section II provides an overview of the full system architecture. Sections III–VII describe each major subsystem in detail. Section VIII discusses system integration and reproducibility. Section IX presents experimental results and discussion, and Section X concludes with directions for future work.

II. SYSTEM PIPELINE OVERVIEW

A single launch file triggers the entire three-phase lifecycle, allowing the system to transition automatically. The three phases are:

- 1) **Exploration (Lap 1)**: The vehicle performs one full lap of the track using a wall-following reactive controller

while simultaneously building an occupancy grid map via RTAB-Map SLAM.

- 2) **Transition (≈ 30 s)**: Upon completing the exploratory lap, the map is automatically saved, SLAM and reactive nodes are terminated, the particle filter is initialized with the new map, and the centerline extractor and raceline optimizer run as one-shot processes.
- 3) **Racing (Lap 2+)**: The vehicle switches to Pure Pursuit control, following the optimized raceline at adaptively increasing speed.

This phased architecture decouples mapping from racing and allows each subsystem to initialize with full information. Figure 1 illustrates the system architecture.

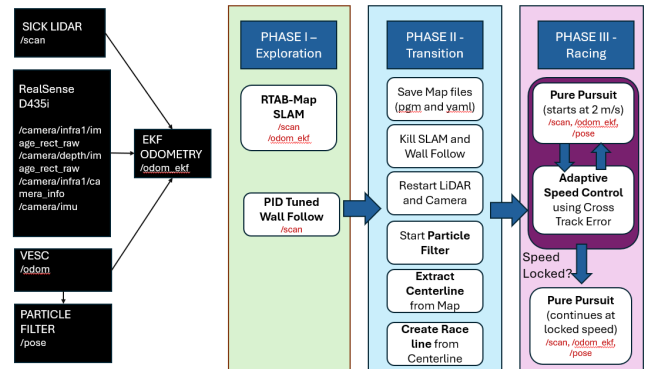


Fig. 1. System pipeline block diagram. Phase 1: SLAM exploration with wall-following. Phase 2: automated transition (saving the map, killing the running nodes, restart LiDAR and camera, particle filter init, centerline extraction, raceline generation). Phase 3: Pure Pursuit Racing with Adaptive Speed Control.

III. REACTIVE EXPLORATION: WALL FOLLOWING

Exploratory mapping is driven by a right-wall following PID controller. We opted for this reactive strategy over Follow-the-Gap methods to ensure greater lateral stability and to minimize the need for extensive environmental calibration. Unlike gap-following logic, which often necessitates precise hyperparameter calibration to dampen oscillations, wall-following utilizes the environment’s inherent geometry as a persistent reference for smoother lateral control.

This approach assumes a closed-circuit environment and a counter-clockwise traversal. A recognized limitation of this

local planner is a tendency to clip interior corners in sharp 90-degree turns, as the controller optimizes for a constant lateral offset rather than trajectory curvature.

A. Lateral Distance Estimation

To achieve stable tracking, the system estimates the vehicle's current and future state relative to the wall using two LiDAR range returns: a perpendicular beam b (at -90° relative to the heading) and a forward-oriented beam a (at an angle θ from b , where $\theta = 50^\circ$).

The orientation of the wall relative to the vehicle, denoted by α , is derived as:

$$\alpha = \arctan\left(\frac{a \cos \theta - b}{a \sin \theta}\right) \quad (1)$$

The instantaneous perpendicular distance D_t is calculated as $D_t = b \cos \alpha$. To compensate for the vehicle's forward velocity and system latency, we project the lateral distance over a lookahead horizon L :

$$D_{t+1} = D_t + L \sin \alpha \quad (2)$$

In this implementation, L is set to 3.0 m. The resulting control error e , defined as the difference between the projected distance and the setpoint $d_{\text{desired}} = 0.8$ m, is expressed as:

$$e = D_{t+1} - d_{\text{desired}} \quad (3)$$

B. Speed Scheduling and Signal Conditioning

The steering command u is computed via the PID law and subsequently processed through an Exponential Moving Average (EMA) filter to mitigate high-frequency sensor noise. To maintain vehicle stability and ensure high-fidelity SLAM data during the exploratory lap, we implement a discrete speed scheduling law. Longitudinal velocity v is modulated as a function of the filtered steering magnitude $|u_f|$:

$$v = \begin{cases} v_{\max} & |u_f| < 0.10 \\ 0.75 v_{\max} & 0.10 \leq |u_f| < 0.30 \\ v_{\min} & 0.30 \leq |u_f| < 0.50 \\ 0.70 v_{\min} & |u_f| \geq 0.50 \end{cases} \quad (4)$$

For the mapping phase, we set $v_{\max} = v_{\min} = 1.0$ m/s. This conservative velocity profile minimizes dynamic tire slip and motion blur, facilitating more accurate scan matching for SLAM.

IV. MAPPING: SLAM

Simultaneous localization and mapping during the exploratory lap is performed using RTAB-Map (Real-Time Appearance-Based Mapping) [4], a graph-based SLAM framework that supports multi-modal sensor fusion with visual loop closure detection. RTAB-Map was selected over 2D-only alternatives (e.g., SLAM Toolbox) because its visual bag-of-words loop closure detector provides robust map correction even when the LiDAR alone produces ambiguous scan matches in symmetric or featureless corridors.

A. Sensor Configuration

The mapping stack fuses three sensor streams:

- **SICK 2D LiDAR**: primary range sensor providing 360° planar scans; used by RTAB-Map's ICP odometry node for scan-to-scan matching and occupancy grid generation.
- **Intel RealSense D435i (stereo + RGB)**: provides visual features for bag-of-words loop closure and stereo depth for 3D obstacle awareness.
- **Intel RealSense D435i (IMU)**: the D435i's onboard IMU serves as the primary inertial source, fused with wheel odometry via an Extended Kalman Filter (EKF) to produce the `/odom_ekf` odometry topic consumed by RTAB-Map.

The EKF fuses wheel encoder odometry from the VESC motor controller with IMU measurements from the D435i, providing drift-corrected odometry that is significantly more reliable than encoder-only estimates during hard cornering. Critically, the D435i IMU is used as the primary inertial source rather than the vehicle's onboard IMU, which exhibited excessive noise during testing. The EKF runs at the IMU rate and publishes to `/odom_ekf`, which serves as the sole odometry input to RTAB-Map, ensuring that both scan matching and loop closure operate on consistent, low-drift pose estimates.

B. Map Parameters and Output

The occupancy grid is generated at a resolution of 0.05 m/cell. Cells are classified as occupied, free, or unknown using thresholds of 0.65 and 0.196 for the occupied and free thresholds, respectively, matching the particle filter's expected map format.

At the end of the exploratory lap, the map is saved automatically by the pipeline state machine—no human intervention is required. The map is written to a `.pgm/.yaml` pair and simultaneously copied to the particle filter's map directory, making it immediately available for localization initialization in the transition phase.

Loop closure during the single exploratory lap corrects accumulated drift and produces a metrically consistent map. Figure 2 shows a representative SLAM map with the extracted centerline overlaid.

V. PATH PLANNING: CENTERLINE EXTRACTION AND RACELINE OPTIMIZATION

Once the SLAM map is available, the centerline extraction module processes the occupancy grid to produce a smooth, closed-loop racing trajectory. The pipeline operates in the following stages.

A. Free-Space Segmentation

The occupancy grid is converted to a binary free-space mask via grayscale thresholding. Cells below the occupancy threshold are classified as free; all others are treated as obstacles or unknown.

B. Distance Transform and Ridge Detection

The Euclidean distance transform $D(x, y)$ is computed over the free-space mask. Skeleton points are selected from cells satisfying

$$D(x, y) \geq \alpha D_{\max}, \quad (5)$$

where $D_{\max}$ is the maximum distance value in the map and $\alpha \in (0, 1)$ is a tunable ridge threshold. This criterion retains only those cells that lie near the geometric center of the free corridor, naturally producing a track centerline.

C. Graph Construction and Pruning

The skeleton is represented as a graph where each skeleton pixel is a node and edges connect 8-connected neighbors. Dead-end branches caused by noise or map artifacts are deleted iteratively by removing leaf nodes (degree-1 vertices) until no leaves remain. Cycle detection is then applied to recover the single closed-loop path that represents the full track.

D. Spline Fitting and Waypoint Sampling

A periodic cubic B-spline is fitted to the ordered set of skeleton points forming the closed loop. The spline ensures C^2 continuity, producing smooth curvature that is critical for high-speed tracking. Waypoints and heading angles are sampled at uniform arc-length intervals along the spline. A smoothing factor of 2000 is applied to suppress high-frequency artifacts introduced by SLAM map noise. Finally, all coordinates are converted from pixel space to world coordinates using the map resolution and origin parameters stored in the map metadata.

E. Raceline Optimization

After centerline extraction, the pipeline runs a raceline optimizer that shifts waypoints laterally within the drivable corridor to minimize total path length. The optimization is formulated as a quadratic program (QP): each centerline point p_i is offset by a scalar α_i along its unit normal $\mathbf{n}_i$, giving a new point $p'_i = p_i + \alpha_i \mathbf{n}_i$. The objective minimizes the sum of squared consecutive segment lengths,

$$\min_{\alpha} \sum_i \|(p_{i+1} + \alpha_{i+1} \mathbf{n}_{i+1}) - (p_i + \alpha_i \mathbf{n}_i)\|^2, \quad (6)$$

subject to box constraints $\alpha_i \in [-(w_{R,i} - m), w_{L,i} - m]$, where $w_{L,i}$ and $w_{R,i}$ are the left and right half-widths to the nearest wall at point i , sampled by ray-casting on the occupancy grid, and m is a configurable safety margin (default 0.5 m). The resulting symmetric positive semi-definite QP is solved with OSQP via `cvxpy`. The optimized raceline is written back to the same waypoint file consumed by the Pure Pursuit controller, so no additional reconfiguration is required. If optimization fails or times out (120 s limit), the system falls back transparently to the unoptimized centerline.

Figure 2 shows the extracted centerline overlaid on the SLAM-generated occupancy grid.

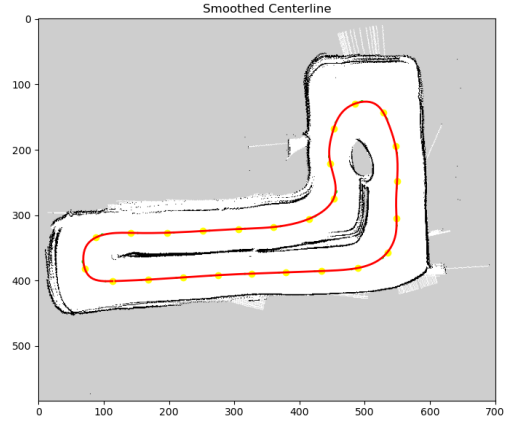


Fig. 2. Extracted centerline overlaid on the SLAM map. Free space is shown in white, occupied walls in black, and unknown cells in gray.

VI. LOCALIZATION: PARTICLE FILTER

Once the transition phase begins, the SLAM localization mode is replaced by a particle filter for efficient real-time localization against the saved static map. The particle filter implementation from the F1TENTH course stack is used, launched via `localize_launch.py` from the `particle_filter` package.

A. Initialization

The particle filter is initialized programmatically at the map origin $(0, 0, 0)$ by publishing an `/initialpose` message with position covariance $\sigma_x^2 = \sigma_y^2 = 0.25 \text{ m}^2$ and heading covariance $\sigma_\theta^2 = 0.1 \text{ rad}^2$. This message is published 30 times over 3 s to ensure the particle filter receives and processes the initial pose before the Pure Pursuit controller begins issuing drive commands.

The initialization is set at the map origin because the vehicle begins and ends the exploratory lap at the same physical location, which was defined as the map origin at SLAM initialization time. This eliminates the need for a global localization phase.

B. Sensor and Motion Models

The particle filter uses the LiDAR scan as the primary observation for likelihood weighting, comparing each particle's predicted scan against the saved occupancy grid via a beam model. Particle propagation uses an odometry motion model driven by the EKF-fused odometry on `/odom_ekf`.

C. Failure Modes

Two failure modes were observed during testing. First, if the SLAM map contains significant artifacts or loop closure errors, the particle filter may fail to converge because the map does not accurately reflect the physical track geometry. Second, if the EKF odometry diverges during the transition (e.g., due to sensor restart latency), the particle filter may initialize with incorrect motion model parameters. Both failure modes are detectable by monitoring the particle weight distribution, but no automated recovery was implemented in the current system.

VII. CONTROL: PURE PURSUIT

Following successful localization initialization, the Pure Pursuit controller takes over as the primary drive policy. The controller is implemented in the `pure_pursuit_reactive` package and launched by the pipeline state machine with the waypoint file produced by the centerline and raceline pipeline.

A. Geometric Tracking Law

Pure Pursuit computes a steering angle that drives the vehicle toward a lookahead point on the reference trajectory. Given the lookahead point at distance ℓ_d from the vehicle, the required steering angle is

$$\delta = \arctan\left(\frac{2L_{wb} \sin \alpha}{\ell_d}\right), \quad (7)$$

where L_{wb} is the vehicle wheelbase and α is the heading error to the lookahead point [6]. The lookahead distance ℓ_d is fixed for a given speed setting; in practice it was tuned per track to balance responsiveness against oscillation.

B. Adaptive Speed Control

A key contribution of the implementation is the adaptive speed controller embedded in the pipeline state machine. Rather than requiring manual speed tuning per track, the system automatically discovers the maximum sustainable speed through closed-loop experimentation:

- 1) Racing begins at a fixed `start_speed` (2.0 m/s).
- 2) After each lap, the average cross-track error (CTE) is computed as the mean minimum distance from the vehicle's sampled positions to the waypoint set.
- 3) If average CTE < threshold (0.21 m), the speed is incremented by `speed_step` (0.4 m/s) and communicated to the Pure Pursuit node via ROS2 `set_parameters` without restarting the controller.
- 4) If average CTE $\geq$ threshold, the system rolls back to the last good speed, locks it, and runs one confirmation lap before stopping.
- 5) A hard cap of 6.0 m/s and a maximum of 15 racing laps prevent unbounded operation.

This approach eliminates the need for manual speed tuning and automatically adapts to track geometry, since tracks with tighter corners or lower-quality SLAM maps will naturally produce higher CTE at lower speeds, causing the controller to lock at a more conservative speed.

The minimum speed is fixed at 1.0 m/s; the Pure Pursuit node schedules speed internally based on curvature between this minimum and the current `max_speed` set by the adaptive controller.

VIII. SYSTEM INTEGRATION

The full pipeline is launched from a single Bash script (`final.sh`) that sequentially brings up the LiDAR, camera and EKF, RTAB-Map SLAM, wall-following controller, and the Racing Blind state machine node (`final.py`). All phase

transitions are handled entirely within the Python state machine, which monitors lap completion via a position-based gate detector and orchestrates subprocess management for each phase transition.

A. Lap Detection

Lap completion is detected using an origin-gate approach: the vehicle is considered to have completed a lap when (1) it has traveled at least 3.0 m from its starting position (`was_far` flag), (2) its current distance to the map origin falls below 0.8 m (`lap_close_dist`), and (3) the total lap odometry exceeds the minimum lap distance. This approach is robust to initial startup jitter and does not require track-specific gate placement.

B. Transition Procedure

Upon SLAM lap completion, the state machine executes the following sequence automatically:

- 1) Save the latest `/map` message as a `.pgm/.yaml` pair and copy it to the particle filter maps directory.
- 2) Kill RTAB-Map, ICP odometry, wall-following, and all sensor launch nodes.
- 3) Restart the LiDAR and camera/EKF stack (3 s + 5 s delays for hardware initialization).
- 4) Launch the particle filter and publish the initial pose 30 times.
- 5) Run `centerline_only.py` (timeout 60 s), then `optimize.py` (timeout 120 s).
- 6) Load the resulting waypoints and launch the Pure Pursuit controller.

The total transition time is approximately 30 s, dominated by hardware restart delays rather than computation—centerline extraction and raceline optimization complete in well under 10 s for the map sizes tested.

C. Robustness Considerations

Several failure modes were identified and mitigated during development. If the SLAM map is poor (e.g., due to loop closure failure or excessive wall contact during exploration), the centerline extraction script may fail to find a valid closed loop, in which case the pipeline halts rather than proceeding with a corrupt waypoint set. If raceline optimization fails or times out, the system falls back to the unoptimized centerline transparently. If the particle filter fails to converge, the Pure Pursuit controller may begin tracking with poor localization, leading to large CTE values that cause the adaptive speed controller to lock at the minimum speed. Pipeline state and per-lap metrics are published in real time on `/racing_blind/state` and `/racing_blind/lap_info` at 10 Hz, reporting phase, speed, position, CTE, and lap distance.

D. Reproducibility

The complete pipeline runs on ROS2 Humble on Ubuntu 22.04. Key dependencies are: `rtabmap_ros` (RTAB-Map ROS2 wrapper), the `FITENTH` `particle_filter` package, `pure_pursuit_reactive`, `sensors_bringup`,

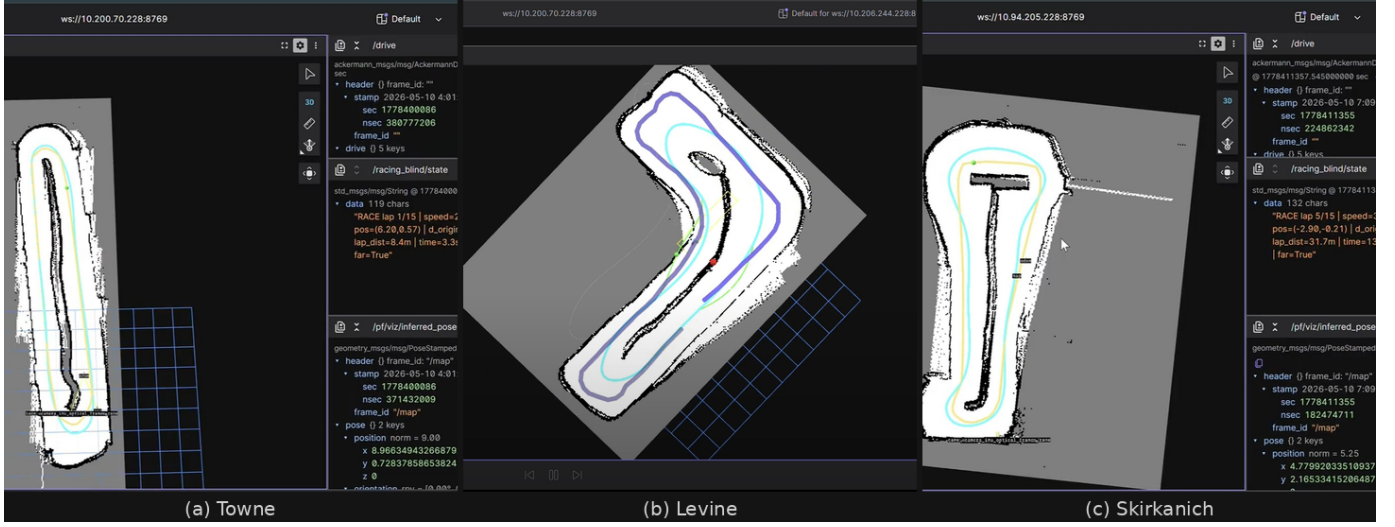


Fig. 3. Foxglove visualizations across all three test tracks during the Pure Pursuit racing phase. (a) Towne: locked speed 3.0 m/s. (b) Levine: RACE lap 1, speed 2.5 m/s, avg CTE 0.112 m. (c) Skirkanich: speed locked at 3.2 m/s after lap 5, avg CTE 0.247 m. White = free space, black = walls; cyan = centerline, yellow/purple = optimized raceline.

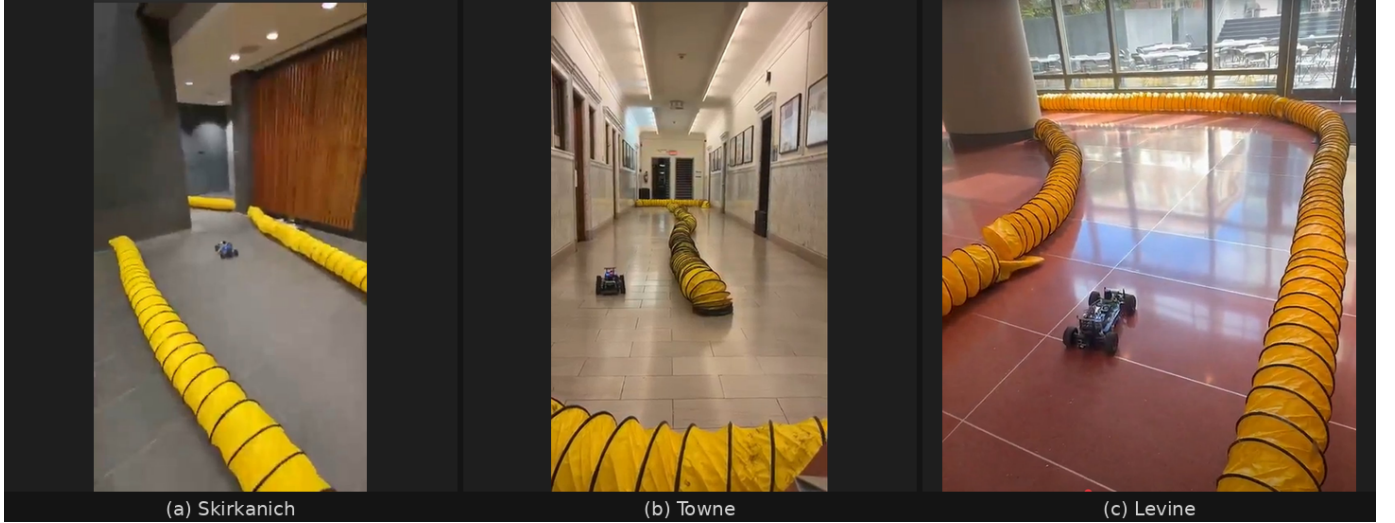


Fig. 4. F1TENTH vehicle operating autonomously during the Pure Pursuit racing phase. (a) Skirkanich Hall, locked speed 3.2 m/s. (b) Towne Building, locked speed 3.0 m/s. (c) Levine Hall, locked speed 4.0 m/s. Track boundaries are defined by foam tube obstacles.

fltenths_stack (LiDAR and VESC drivers), cvxpy with the OSQP backend, and standard scientific Python (numpy, scipy, Pillow). The entire pipeline is launched with a single command:

```
bash final.sh
```

All tunable parameters—SLAM laps, start speed, speed step, CTE threshold, wall margin, and map name—are declared at the top of `final.sh` and passed to the state machine node at launch. No source-code edits are required to adapt to a new track. The RTAB-Map database is cleared automatically at each run to ensure a fresh map is built from scratch.

IX. RESULTS AND DISCUSSION

A. Experimental Setup

All experiments were conducted on a physical F1TENTH platform equipped with a SICK TiM571 2D LiDAR (270° field of view, 0.33° angular resolution), an Intel RealSense D435i RGB-D camera and IMU (front-mounted), and a NVIDIA Jetson Orin Nano Super as the onboard compute platform. The pipeline ran entirely onboard with no offboard computation. No simulation was used at any stage.

Three tracks were used for evaluation: Levine Hall corridor (largest track), Towne Building corridor, and Skirkanich Hall corridor. Track boundaries were defined by foam tube obstacles. All three tracks were treated as completely unknown at the start of each trial—no prior maps or human-assisted

initialization were used. Each track received a complete end-to-end run: one SLAM exploration lap at 1.0 m/s, followed by automated transition, followed by adaptive racing laps until the speed controller locked. The raceline optimizer fallback to unoptimized centerline was also exercised on a degraded SLAM map to confirm the pipeline continues gracefully rather than halting.

B. Qualitative Results

The pipeline successfully completed the full explore-to-race sequence on all three tracks without any track-specific parameter changes. The SLAM phase consistently produced usable occupancy grid maps within approximately 40 s at the 1.0 m/s exploration speed. Centerline extraction and raceline optimization completed within the 30 s transition window in all trials. Figures 3 and 4 show Foxglove visualizations and physical vehicle operation across all three tracks.

C. Quantitative Results

Table I summarizes the performance across all three tracks. SLAM lap times reflect the single exploratory lap at 1.0 m/s wall-following speed. The locked racing speed is the final speed at which the adaptive controller converged, determined automatically by the CTE feedback loop.

TABLE I
PIPELINE PERFORMANCE ACROSS THREE TRACKS

Track	SLAM Lap (s)	Transition (s)	Locked Speed (m/s)
Levine	~40	~30	4.0
Towne	~40	~30	3.0
Skirkanich	~40	~30	3.2

The Levine track achieved the highest locked speed (4.0 m/s) owing to its wider corridors, which give the shortest-path optimizer more lateral freedom and produce a smoother raceline with lower CTE at a given speed. Towne and Skirkanich locked at lower speeds (3.0 and 3.2 m/s respectively), consistent with their tighter turn geometries. The adaptive controller converged to these speeds without any manual tuning, demonstrating that the CTE-based feedback loop generalizes across track geometries.

D. Discussion

The primary limitation of the current system is sensitivity to SLAM map quality. On tracks where the wall-following controller caused minor wall contact at sharp corners, the resulting map artifacts occasionally caused the centerline extractor to produce waypoints that cut corners, leading to elevated CTE in the racing phase. A more aggressive corner slow-down in the wall-following controller would reduce this sensitivity.

The particle filter initialization is the second main reliability bottleneck. Publishing the initial pose programmatically at the map origin proved reliable in all three tested tracks, but this approach assumes the vehicle returns to within 0.8 m of its starting position at the end of the exploratory lap. Significant starting-position drift would require a global localization step.

The 30 s transition window is dominated by hardware restart delays (LiDAR: 3 s, camera and EKF: 5 s, particle filter: 5 s) rather than computation. Eliminating unnecessary restarts of hardware drivers that remain valid across phases would reduce this to under 15 s.

The current Pure Pursuit implementation utilizes static gains, which limits the system’s ability to minimize lap times compared to a manually tuned or adaptive controller. The current speed profile is effectively capped by the lack of a robust adaptive speed controller. Furthermore, the existing obstacle avoidance logic is limited to low-speed regimes; a race-ready pipeline requires a more robust avoidance and overtaking strategy capable of high-speed maneuvers.

The limitations of the current adaptive speed control further hinder plug-and-play modularity. Refactoring this component into a more generalized framework would allow for seamless integration of advanced control architectures, such as Model Predictive Control (MPC) or Model Predictive Path Integral (MPPI) control, without requiring significant system-wide reconfiguration.

X. CONCLUSION AND FUTURE WORK

This work presents a unified, fully autonomous racing pipeline for the FITENTH platform that requires no prior map or human initialization. By chaining wall-following SLAM, automatic centerline extraction with raceline optimization, particle filter localization, and adaptive Pure Pursuit control into a single launch sequence, the system demonstrates reliable generalization across three distinct unknown tracks.

Several avenues exist for enhancing system performance. Regarding the mapping phase, augmenting the wall-following controller with curvature-aware speed reduction would mitigate map artifacts at sharp 90-degree turns. In terms of control and dynamics, replacing the Pure Pursuit algorithm with a Model Predictive Control (MPC) or Model Predictive Path Integral (MPPI) controller would allow for tighter trajectory tracking and better handling of non-linear vehicle dynamics at high speeds. Furthermore, developing a generalized adaptive framework for real-time gain tuning would enable the vehicle to optimize performance across successive laps.

Transition overhead can be significantly reduced by streamlining hardware handoffs between phases and removing unnecessary initialization sequences. The core challenge lies in the engineering feasibility of optimizing startup routines and achieving high-speed state switching without compromising system stability. Ultimately, extending this optimized pipeline to multi-agent scenarios—where the ego vehicle must detect and react to dynamic opponents in real-time—represents a critical step toward a robust, race-ready autonomous system.

REFERENCES

- [1] O’Kelly, M., Zheng, H., Karthik, D., and Mangharam, R., “FITENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning,” in *Proc. NeurIPS 2019 Competition and Demonstration Track*, 2020.
- [2] J. Betz et al., “AMZ Driverless: The Full Autonomous Racing System,” *IEEE Intelligent Vehicles Symposium (IV)*, 2019.

- [3] M. Wischniewski et al., “ForzaETH Race Stack: Scaled Autonomous Head-to-Head Racing on Full-Scale Race Tracks,” 2023.
- [4] M. Labbé and F. Michaud, “RTAB-Map as an Open-Source Lidar and Visual Simultaneous Localization and Mapping Library for Large-Scale and Long-Term Online Operation,” *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.
- [5] G. Williams et al., “Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving,” *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.
- [6] R. Coulter, “Implementation of the Pure Pursuit Path Tracking Algorithm,” Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.